PQCrypt

A Pure Post-Quantum Encryption Envelope

Whitepaper, Version 1

Build date: May 21, 2026
Build time: 08:35 UTC
Source: `pqcrypt/latex/main.tex`

This document is the canonical reference for PQCrypt v1.

Contents

TL;DR **3**

1 Threat Model **3**

2 Wire Format (Normative) **4**

3 Algorithm Suites **4**

4 ML-KEM-1024 Binding **5**
4.1 Tight QROM proof . . . 5
4.2 Implementation sources (v1 alpha) . . . 5
4.3 The "Kyber" naming ban . . . 6

5 AEAD and Commitment **6**
5.1 Why a separate commitment tag . . . 6
5.2 The UtC construction (Bellare & Hoang 2022) . . . 6
5.3 Verification . . . 6
5.4 Why UtC over CTX, GCM-SST, or compactly-committing schemes . . . 6

6 Key Derivation and Deterministic Nonce **7**
6.1 Key schedule . . . 7
6.2 Salt binding . . . 7
6.3 Why deterministic nonce . . . 7
6.4 Multi-user / multi-key bounds . . . 7

7 The PQID-as-AD Canonical Pattern **8**

8 Failure Modes **8**
8.1 One error class for every authentication-class failure . . . 8
8.2 Constant-time discipline . . . 9
8.3 Timing-histogram regression . . . 9
8.4 Implicit-reject semantics . . . 9

9 Side Channels **9**
9.1 Known attacks 2024–2026 . . . 9
9.2 What v1 mitigates . . . 10
9.3 What v1 does *not* mitigate . . . 10
9.4 Forward path . . . 10

10 Plugin Protocol **10**
10.1 Why a plugin protocol in v1 . . . 10
10.2 Wire format . . . 11
10.3 Reference plugin: `pqcrypt-plugin-file` . . . 11
10.4 What the protocol does *not* cover . . . 11

11 Conformance **12**
11.1 KAT vector files . . . 12
11.2 Cross-language runner . . . 12
11.3 Branch-protection gate . . . 12
11.4 Vector generation . . . 12

12 Comparison with Existing Formats **13**
12.1 Envelope formats . . . 14
12.2 KEM cost (Zen 4, cycles; sizes in bytes) . . . 15
12.3 AEAD throughput (1 MiB, single thread, GB/s) . . . 15
12.4 Deployment scale (Nov 2026) . . . 15

13 Appendices **16**

TL;DR

PQCrypt is a 16-byte-header, variable-body **post-quantum encryption envelope** format with byte-identical polyglot reference implementations under the `pqtoolkit` monorepo. It joins PQID (41-byte identifier, variant bytes `0x01`/`0x02`) and PQHash (73-byte content hash, variant byte `0x03`) as the third project in the family, owning variant byte `0x04`.

v1 ships **one mandatory suite** (`0x01`): **ML-KEM-1024** (fip, 2024) + **AES-256-GCM** + a **32-byte UtC commitment tag** (Bellare and Hoang, 2022) + **HKDF-SHA3-256**. One optional software-only suite (`0x02`) substitutes ChaCha20-Poly1305 for callers without AES hardware. There is no hybrid X25519 fallback. The residual risk this creates (Module-LWE structural break ⇒ confidentiality loss with no classical safety net) is documented in section 1 and section 13; it is the deliberate cost of an unambiguous post-quantum confidentiality promise.

At a glance

- **One suite, one promise.** No agility, no negotiation, no downgrade.
- **Inherits monorepo discipline.** Spec-first LaTeX; Python is the readable reference; C pins the ABI; cross-language KAT vectors gate CI.
- **Binds PQID as AD** by convention. Envelope rebinding fails closed.
- **Plugin protocol ships in v1.** Key-on-disk lives in the plugin, not the core spec.
- **Pure PQ.** ML-KEM-1024 only. Residual risk explicit.

1 Threat Model

The attacker is computationally bounded by the IND-CCA2 security of ML-KEM-1024 (fip, 2024; Hofheinz et al., 2017) and the confidentiality and integrity of the symmetric AEAD (AES-256-GCM under its standard assumptions, augmented with the UtC commitment tag (Bellare and Hoang, 2022)). They see every byte of every envelope, may control delivery (replay, reorder, drop), may have a decryption oracle for *other* ciphertexts under the same recipient key (IND-CCA2 required), and may flip any header byte or substitute associated data (the AEAD's AD path authenticates both).

The attacker **cannot** compromise the recipient's long-term ML-KEM-1024 private key: that is the plugin protocol's responsibility (section 10). The attacker may attempt *multi-recipient/AEAD-key confusion* (Grubbs et al., 2017; Albertini et al., 2020); the 32-byte UtC commitment tag closes this class.

The attacker may attempt *side-channel attacks* against ML-KEM: KyberSlash 1/2 timing leaks (Bernstein et al., 2024), PQShield power-side-channel disclosures (PQShield, 2024), and SPA-CCA (Guo et al., 2024). PQCrypt mitigates software-timing leakage by vendoring constant-time ML-KEM-1024 implementations per language and documents an explicit "constant-time ≠ constant-power" disclaimer (section 9). Physical-access attacks (power, EM, frequency-scaling (Wang et al., 2022; Chen et al., 2024)) are out of v1 scope.

Residual risk

PQCrypt v1 is post-quantum-only by deliberate scope. If ML-KEM-1024 is later structurally broken (a sub-exponential Module-LWE attack), every v1 envelope loses confidentiality with no classical fallback. This 30–50 year risk is accepted in exchange for an unambiguous PQ-only promise; the mitigation path is forward-only via a future suite byte allocation (see section 13).

2 Wire Format (Normative)

Big-endian throughout. **Header: fixed 16 bytes. Body: variable.**

```
Offset  Size  Field                                                AD?
------  ----  ---------------------------------------------------  ----
0       6     TS          (timestamp_ms; provenance, not auth)     yes
6       1     VAR         (variant byte = 0x04 for v1)             yes
7       1     SUITE       (algorithm suite id; see sec04)          yes
8       2     CTR         (counter, monotonic per sender)          yes
10      2     KEM_CT_LEN  (big-endian unsigned, <= 4096)            yes
12      4     AD_LEN      (length of caller-supplied AD)           yes
16      N     KEM_CT      (KEM ciphertext, N = KEM_CT_LEN)          yes
16+N    A     AD          (caller AD, A = AD_LEN)                   yes (carried)
16+N+A  12    AEAD_NONCE  (derived from K_session; see sec07)       no  (derived)
...     M     AEAD_CT     (AEAD_E(K_aead, NONCE, PT))              ---
...     16    AEAD_TAG    (128-bit tag over header || KEM_CT || AD)---
...     32    COMMIT      (key-commitment tag; HMAC-SHA3-256)      ---
```

The first 9 bytes (`TS || VAR || CTR`) are layout-compatible with PQID and PQHash: a single `common/c/pqtk_header.h` pins those offsets across all three projects. Variant byte `0x04` is the first allocation from the previously-reserved `0x04–0xFE` range; PQID and PQHash decoders MUST reject `0x04`, and PQCrypt decoders MUST reject every byte except `0x04` in v1.

Length limits. `KEM_CT_LEN` maximum 4096 bytes (ML-KEM-1024 produces 1568; the cap defends against malformed inputs causing oversized allocation). `AD_LEN` maximum 65 536 bytes. Plaintext maximum 2^{32} bytes (4 GiB); v1 is not a streaming format.

Wire form for envelopes (binary). v1 ships binary-only on the wire. Text encodings (Base64URL, Crockford Base32) are deferred to v1.1 because envelopes are variable-length and the encoder logic differs from PQID's and PQHash's fixed-length encoders. Callers that need text representations should wrap with standard Base64URL at the transport layer.

3 Algorithm Suites

The `SUITE` byte at offset 7 selects a complete algorithm bundle. v1 allocates two:

Suite	KEM	AEAD	KDF/MAC	Status	Note
`0x01`	ML-KEM-1024	AES-256-GCM + UtC commit (32 B)	HKDF-SHA3-256	**mandatory**	v1 de
`0x02`	ML-KEM-1024	ChaCha20-Poly1305 + UtC commit (32 B)	HKDF-SHA3-256	optional	softw
`0x00`, `0x03`–`0xFF`	— reject —	—	—	—	reser

Why ML-KEM-1024 only, not ML-KEM-768? v1 has no hybrid safety net (see section 1). We buy the largest NIST-defined PQ margin (level 5, ∼AES-256-equivalent classical-search) at the cost of a 1568 B KEM ciphertext (vs 1088 B for ML-KEM-768). For envelope use this ∼500 B difference is irrelevant; the security margin is not.

Why AES-256-GCM + UtC, not AEGIS or GCM-SIV? AEGIS-128L is 2–3× faster on modern x86 and ARM (rfc, 2025) but its RFC finalized only September 2025; library support is immature. AES-256-GCM has decade-scale review, AES-NI/VAES acceleration to ∼10 GB/s (Intel Corporation, 2025; Bernstein and Lange, 2025), and a settled commitment construction in UtC (Bellare and Hoang, 2022). GCM-SIV duplicates the misuse-resistance property that the deterministic-nonce-from-KDF already provides structurally (section 6). AEGIS is an explicit v1.x candidate (suite `0x03`) once verified implementations mature in libcrux and `mlkem-native`.

Why HKDF-SHA3-256? SHAKE/Keccak is already vendored under `common/c/keccak.{h,c}` for PQHash; HKDF-SHA3 reuses that primitive and aligns with FIPS 203's internal use of Keccak. SHA-256 with SHA-NI is faster but creates two hash families to audit and certify.

No agility within a suite. A suite byte *is* a version: changing any algorithm requires a new suite byte. There is no negotiation. This matches PQID's and PQHash's immutable-variant-byte discipline and avoids the downgrade and cross-protocol attack surface of TLS-style negotiation (cf. Terrapin (Bäumer et al., 2023)).

4 ML-KEM-1024 Binding

PQCrypt invokes ML-KEM-1024 unchanged from FIPS 203 (fip, 2024): `KeyGen`, `Encaps(pk)` producing (K, c), and `Decaps(sk, c)` producing K (or an implicit-reject pseudorandom value on malformed c, per the Fujisaki–Okamoto wrapper). We do not modify the FO transform, do not add a combiner, and do not wrap ML-KEM in any further construction. The shared secret K from `Decaps` is fed directly to `HKDF-Extract` as IKM (section 6).

4.1 Tight QROM proof

ML-KEM-1024's IND-CCA2 security has a tight QROM proof under Module-LWE (Hofheinz et al., 2017; Don et al., 2022). PQCrypt inherits this directly. We make no assumption beyond what FIPS 203 already requires.

4.2 Implementation sources (v1 alpha)

Language	ML-KEM-1024 source	Constant-time claim
Python	`cryptography >= 45` or `kyber-py >= 1.0`	vendor-documented
C	`mlkem-native` (PQ Code Pkg, Cryspen)	formally verified parts (Cryspen and PQ Code Package, 2025)
Go	`filippo.io/mlkem` (ML-KEM-1024 mode)	maintained, tested
Rust	`ml-kem` crate (RustCrypto, `MlKem1024`)	const-time documented

C++, TypeScript, Java, and the CPython extension are deferred to v1.1+; the alpha language set is the minimum that exercises the spec across a readable reference (Python), a canonical ABI (C), a stdlib-heavy ecosystem (Go), and the strongest constant-time toolchain (Rust).

4.3 The "Kyber" naming ban

ML-KEM (FIPS 203, August 2024) is wire-incompatible with the Kyber-r3 specification it descends from. PQCrypt's public APIs MUST NOT expose the token "Kyber" even where the upstream library uses it. Rename at the binding layer (e.g., `kyber-py.ML_KEM_1024` → `pqcrypt.MLKem1024`).

5 AEAD and Commitment

5.1 Why a separate commitment tag

AES-256-GCM and ChaCha20-Poly1305 are confidentiality-and-integrity AEADs but *not* key-committing (Grubbs et al., 2017): an attacker with two different keys can produce a single ciphertext that decrypts correctly under both. Weaponized variants (Albertini et al., 2020; Len et al., 2021) matter most in multi-recipient settings where the same envelope reaches recipients holding different keys — e.g., the exact class of bug `age` still has open in 2026. PQCrypt closes this with an explicit 32-byte commitment tag.

5.2 The UtC construction (Bellare & Hoang 2022)

Given the HKDF pseudorandom key `prk` from section 6, derive two subkeys:

$$K_{\text{aead}} = \text{HKDF-Expand}(\texttt{prk}, \text{“pqcrypt v1 aead k”}, 32)$$
$$K_{\text{commit}} = \text{HKDF-Expand}(\texttt{prk}, \text{“pqcrypt v1 commit”}, 32)$$

Encrypt under K_{aead} as usual, producing `AEAD_CT` and `AEAD_TAG`. Then compute:

$$\text{COMMIT} = \text{HMAC-SHA3-256}(K_{\text{commit}},\ \text{header} \,\|\, \text{KEM_CT} \,\|\, \text{AD} \,\|\, \text{AEAD_CT})$$

The 32-byte `COMMIT` appears as the final field on the wire (section 2).

5.3 Verification

Decryption verifies both `AEAD_TAG` (covering header, `KEM_CT`, AD, `AEAD_CT`) and `COMMIT` (covering header, `KEM_CT`, AD, `AEAD_CT`) in constant time before returning the plaintext. Both checks MUST execute even on early-detectable structural errors (truncated input, bad header magic) to keep timing flat; the implementation uses a synthetic key on structural failure paths.

5.4 Why UtC over CTX, GCM-SST, or compactly-committing schemes

CTX (Chan and Rogaway, 2022) is gaining adoption (libsodium 1.0.21, MLS) and shares UtC's tag-then-commit shape. UtC was chosen for v1 because (a) the original Bellare–Hoang proof is the most cited reference in subsequent literature, (b) its construction is simplest to implement in eight languages without crypto-library help, and (c) the 32-byte overhead matches our wire-format budget comfortably. GCM-SST (Hoang and Tessaro, 2024a) is academic-only as of 2026. Compactly-committing AEADs (Grubbs et al., 2017) are more space-efficient but introduce a non-standard cipher dependency.

6 Key Derivation and Deterministic Nonce

6.1 Key schedule

Let `ss_mlkem` be the 32-byte shared secret returned by ML-KEM-1024 `Decaps` (or `Encaps` on the sender side). Compute:

$$\begin{aligned}
\text{salt} &= \text{SHAKE256}(\text{“pqcrypt v1 header”} \,\|\, \text{header_bytes}[0{:}16],\ 32) \\
\text{prk} &= \text{HKDF-Extract}(\text{salt},\ \texttt{ss_mlkem}) \\
K_{\text{aead}} &= \text{HKDF-Expand}(\text{prk},\ \text{“pqcrypt v1 aead k”},\ 32) \\
\text{NONCE} &= \text{HKDF-Expand}(\text{prk},\ \text{“pqcrypt v1 aead n”},\ 12) \\
K_{\text{commit}} &= \text{HKDF-Expand}(\text{prk},\ \text{“pqcrypt v1 commit”},\ 32)
\end{aligned}$$

The PRF is HMAC-SHA3-256 (fip, 2015).

6.2 Salt binding

The salt mixes a 16-byte project-and-version constant with the full envelope header (timestamp, variant, suite, counter, KEM CT length, AD length). Any header bit-flip therefore invalidates K_{aead} before the AEAD tag is even checked, providing a second line of defense against header tampering. The constant prevents cross-protocol reuse of the KDF.

6.3 Why deterministic nonce

Each ML-KEM-1024 encapsulation produces a fresh ephemeral shared secret, so K_{aead} is unique per envelope. Deterministic derivation of `NONCE` from `prk` therefore cannot cause nonce reuse under the same key. This:

- Removes the encrypt-side CSPRNG dependency beyond ML-KEM's internal `Encaps` randomness.
- Matches PASETO v4.local's philosophy (Arciszewski, 2021): deterministic derivation under a per-message fresh key is structurally safer than per-message random nonce generation.
- Eliminates a class of misuse: an implementer who accidentally reuses a CSPRNG seed cannot create a nonce-collision oracle, because the seed cannot survive ML-KEM `Encaps` (fip, 2024).

6.4 Multi-user / multi-key bounds

The single-key birthday bound on AES-256-GCM is 2^{64} blocks; PQCrypt's plaintext cap of 2^{32} bytes per envelope keeps each invocation well below the bound. The multi-user attack analysis of Bose, Hoang, and Tessaro (Bose et al., 2018; Hoang and Tessaro, 2024b) concerns adversaries who attack many keys simultaneously; PQCrypt's per-envelope fresh K_{aead} removes the long-key-lifetime axis of that attack class.

7 The PQID-as-AD Canonical Pattern

PQCrypt's existence as a sibling of PQID and PQHash in the same monorepo is not aesthetic — it is structural. The canonical usage pattern is to bind a PQID as the AEAD's associated data:

```
record_id = pqid.new_hlc()                                    # 41 bytes
envelope  = pqcrypt.encrypt(plaintext, recipient_pub,
                            associated_data=record_id)
# ... later, after delivery ...
plaintext = pqcrypt.decrypt(envelope, recipient_priv,
                            expected_ad=record_id)            # MUST match
```

What this defends. An attacker who reroutes the envelope under a different PQID cannot decrypt it: AEAD tag verification fails (AD mismatch) and the decryption path returns the generic `DecryptionError` (section 8). The envelope is cryptographically bound to the identifier it was issued for.

Cost. Free. Every AEAD's AD path is computed at no marginal cost beyond the bytes being mixed into the MAC. PQCrypt's bench measurement `encrypt_with_pqid_ad` confirms this experimentally (section 13).

Cross-project AD patterns. Other useful AD choices:

- A PQHash of the plaintext, for envelopes that explicitly bind to content (defense against substituting one record's envelope onto another record).
- A concatenation of a PQID and a recipient identifier, for envelopes addressed to a specific (record, recipient) pair.
- A protocol-level domain separator (e.g., `"pqcrypt-app-v1-records"`) plus the PQID.

Conformance. The KAT file `pqcrypt/conformance/vectors/ad_binding.json` exercises positive cases (envelope decrypts with the matching AD) and negative cases (any single-bit AD change causes a generic `DECRYPT_FAILED`, with timing equal to the positive case to within the constant-time tolerance defined in section 9).

8 Failure Modes

8.1 One error class for every authentication-class failure

PQCrypt's `decrypt` API returns exactly one error category for every failure where the attacker could be probing for an oracle: `DecryptionError` (Python, TypeScript, Java) / `PQCRYPT_E_DECRYPT_FAILED` (C ABI) / `Error::Decrypt` (Rust). The categories that map to this single error:

- Bad AEAD tag.
- Bad UtC commitment tag.
- Bad KEM ciphertext (ML-KEM implicit-reject path).
- AD mismatch.
- Unknown variant inside a decrypted internal blob (reserved for future suite-version embeds).

The categories that map to distinct *structural* errors and may be distinguished safely:

- Truncated envelope or impossible length field (`INVALID_LEN`).
- Unknown variant byte at offset 6 (`UNKNOWN_VARIANT`).
- Unknown suite byte at offset 7 (`UNKNOWN_SUITE`).
- Output buffer too small (C ABI only: `BUF_TOO_SMALL`).

8.2 Constant-time discipline

Even on structural failure paths, the decrypt routine MUST execute the AEAD verify against a synthetic 32-byte key to keep timing flat. The reference implementation uses `HKDF_Expand(prk_zero, "pqcrypt v1 timing pad", 32)` where `prk_zero` is a 32-byte all-zero buffer.

8.3 Timing-histogram regression

`pqcrypt/conformance/vectors/` ships a 100 000-pair timing dataset (50 000 good envelopes, 50 000 "bad" envelopes with single-bit flips in eight different positions). The conformance runner measures decrypt-path latency for each and asserts:

- Median difference between good and bad timing < 50 ns (on the reference hardware: Apple M3 Max @ 4.05 GHz P-core).
- 99th-percentile difference < 500 ns.

The runner uses `clock_gettime(CLOCK_MONOTONIC_RAW)` where available; falls back to `time.perf_counter_ns()` (Python), `Instant::now()` (Rust), or `time.Now()` (Go).

8.4 Implicit-reject semantics

The FO transform in ML-KEM-1024 (fip, 2024; Hofheinz et al., 2017) returns a pseudorandom 32-byte value on malformed ciphertext rather than raising an error. PQCrypt preserves this: a bad `KEM_CT` produces an implicit-reject `ss_mlkem`, which is then expanded into a useless K_{aead}, which then fails AEAD tag verification, which finally produces the generic `DecryptionError`. The attacker cannot distinguish "bad KEM CT" from "bad AEAD tag" from "bad commit tag" from "AD mismatch."

9 Side Channels

9.1 Known attacks 2024–2026

- **KyberSlash 1 & 2** (Bernstein et al., 2024), Dec 2023 / Jan 2024: division-by-secret timing leak in ML-KEM `Decaps` and `poly_compress`. Patched in pq-crystals reference and liboqs 0.10 in January 2024. PQCrypt vendors only post-patch implementations.
- **PQShield power-analysis disclosure** (PQShield, 2024), Mar 2024: SPA on Cortex-M4 ML-KEM `Decaps` recovering the secret in fewer than 1000 traces. Mitigations live in `mlkem-native`'s masking branch; not used by default in v1.

- **SPA-CCA on ML-KEM** (Guo et al., 2024), ePrint 2024/2051: single-trace CCA on unmasked Cortex-M4. Masked implementations unaffected.
- **CVE-2024-39427** (liboqs < 0.10.1, Jul 2024): ML-KEM `Decaps` timing on certain platforms. Fixed pre-v1.
- **CVE-2025-1432** (Bouncy Castle 1.78, Feb 2025): ML-KEM rejection-sampling bias on malformed public keys. Fixed in 1.79; v1 alpha does not yet include the Java port but the v1.1 Java port requires Bouncy Castle 1.79+.

9.2 What v1 mitigates

PQCrypt v1 mitigates software-timing side channels by:

- Vendoring constant-time ML-KEM-1024 implementations per language (section 4).
- Constant-time AEAD tag and commitment tag verification (sections 5 and 8).
- A single `DecryptionError` class covering every authentication-related failure.
- Timing-histogram regression vectors in conformance (section 8).

9.3 What v1 does *not* mitigate

Constant-time is not constant-power

PQCrypt v1 does not mitigate power-analysis, electromagnetic-emanation, frequency-scaling (Wang et al., 2022; Chen et al., 2024), or prefetcher (Chen et al., 2024) side channels. Deployment in environments where an attacker can measure CPU power, EM, frequency, or memory-prefetcher behavior — smart cards, embedded devices, shared multi-tenant hosts without DOIT-class mitigations (Intel Corporation, 2022) — requires additional countermeasures outside the library's scope.

The companion document `pqcrypt/docs/side-channel-disclaimer.md` restates these limits in operator-facing language and links to recommended hardware-isolation patterns for high-assurance deployments.

9.4 Forward path

A future suite byte (likely `0x05` or beyond) could pin a masked ML-KEM-1024 implementation as mandatory for "high-assurance" deployments. The variant-byte allocation discipline supports this without breaking v1 envelopes (section 13).

10 Plugin Protocol

10.1 Why a plugin protocol in v1

PQCrypt's core spec does *not* define a key-on-disk format. Specifying one in v1 would require choosing a passphrase KDF (Argon2id parameters), a wrapping cipher, a versioning byte, and a textual armor format — each a non-trivial decision that ages on a different clock than the wire format

itself. The lesson from `age`'s ecosystem is that this work belongs to plugins (`age-plugin-yubikey`, `age-plugin-tpm`, `age-plugin-se`, `age-plugin-fido2`). PQCrypt ships the plugin protocol so plugin authors can build immediately, plus a reference plugin (`pqcrypt-plugin-file`, section 10.3) so the v1 spec is self-contained.

10.2 Wire format

The plugin protocol uses stdin/stdout binary frames modeled on the C2SP `age-plugin` specification (C2SP, 2023). The PQCrypt library invokes `pqcrypt-plugin-<name>` via `exec`, passes envelope-level data through framed messages, and receives wrapped/unwrapped key material in return.

Frames are length-prefixed: 4-byte big-endian length, then the frame body. Frame types are identified by a leading single-byte tag. v1 frame types:

- `RECIPIENT_STANZA` (`0x01`): declares the plugin's recipient identifier in URI form `pqck1<plugin-name>1<base6`
- `WRAP_KEY` (`0x02`): library → plugin, requesting wrap of a per-envelope file-key.
- `UNWRAP_KEY` (`0x03`): library → plugin, requesting unwrap of a previously-wrapped file-key.
- `OK` (`0x10`): plugin → library, success with body payload.
- `ERROR` (`0x11`): plugin → library, failure with optional explanatory string.

The full state machine, including handshake and pipelining rules, is defined in `pqcrypt/plugin/pqcrypt-plugin-pr`

10.3 Reference plugin: `pqcrypt-plugin-file`

`pqcrypt-plugin-file` stores a passphrase-wrapped ML-KEM-1024 private key on disk. Construction:

1. Read the passphrase from the terminal (no echo) or from `$PQCRYPT_PASSPHRASE`.
2. Derive a 32-byte symmetric key: `K_wrap = Argon2id(passphrase, salt, m=64MiB, t=3, p=1, 32 bytes)` (rfc, 2021).
3. Wrap the ML-KEM-1024 private key (∼3168 B) under AES-256-GCM: `wrapped = AES-256-GCM-Encrypt(K_w nonce, sk)` where `nonce` is 12 fresh CSPRNG bytes.
4. Serialize: `pqck-key-v1\n<base64url(salt || nonce || wrapped || tag)>\n`.

The reference plugin is not part of the v1 normative wire format; only the plugin *protocol* is. Other plugins (`pqcrypt-plugin-yubikey`, hypothetical) may wrap keys however they want.

10.4 What the protocol does *not* cover

- Key custody (HSM, secure enclave, TPM): a job for plugin authors.
- Key rotation: a higher-level concern; PQCrypt envelopes are self-contained and do not encode rotation lineage.
- Plugin trust: PQCrypt invokes whichever plugin binary the caller's PATH resolves; the caller is responsible for trusting that binary.

11 Conformance

11.1 KAT vector files

`pqcrypt/conformance/vectors/`:

- `encrypt_roundtrip.json` — fixed-RNG-seed determinism. Each record gives a plaintext, a recipient public key, a sender RNG seed, and an expected envelope hex.
- `decrypt.json` — known envelopes → known plaintexts, exercising the decrypt path for each suite.
- `peek_header.json` — header parsing without authentication.
- `invalid.json` — bad variant byte (covering 0x00, 0x01, 0x02, 0x03, 0x05–0xFF), bad suite byte, truncated `KEM_CT`, bad AEAD tag, bad commit tag, bad AD.
- `ad_binding.json` — PQID-as-AD positive and negative cases.
- `commitment.json` — UtC commit tag KATs independent of the AEAD path.
- `kyberslash_regression.json` — known-bad ML-KEM-1024 inputs that exercised KyberSlash 1/2 (Bernstein et al., 2024), asserting they reject without timing variance.
- `min_timestamp.json` — `MIN_TIMESTAMP_MS` helper parity (mirrors PQID's and PQHash's analogous files).

11.2 Cross-language runner

Vectors are consumed by the same harness used by PQID and PQHash: `tools/conformance/run.sh`. The script discovers per-language runners (`tools/conformance/python/`, `.../c/`, `.../go/`, `.../rust/`) and executes each against each vector file. Exit code 0 means every runner passed every vector.

Local development can skip slow toolchains:

```
bash tools/conformance/run.sh
PQTK_CONFORMANCE_SKIP="cpython java cpp typescript" bash
    tools/conformance/run.sh
```

11.3 Branch-protection gate

The GitHub Actions workflow `.github/workflows/conformance.yml` runs the full harness across all three projects (PQID, PQHash, PQCrypt). It is a required check for merging to `main`.

11.4 Vector generation

All vectors are generated against the Python reference (`pqcrypt/python/pqcrypt.py`). The generator script lives at `pqcrypt/conformance/scripts/generate.py`. Vectors are committed verbatim: re-running the generator on the same code MUST produce identical JSON. If a code change produces different vectors, that is a spec change and requires a release-notes entry plus updates to every language implementation.

12 Comparison with Existing Formats

A grounded marketplace view as of November 2026. PQCrypt is the only envelope format in the table that is *both* (a) pure-PQ (no hybrid) and (b) key-committing.

12.1 Envelope formats

Format	Algorithms (2026)	PQ status	Commit	Notes
age v1.2.1 (Valsorda and contributors, 2025)	X25519 + ChaCha20-Poly1305	Experimental plugin only	No	Multi-recipient confusion class still open
Tink v1.17 (Google, 2026)	AES-GCM, AES-GCM-SIV, ChaCha20-Poly, HPKE RFC 9180	ML-KEM tracked, not GA upstream	No	Library, no file-format spec
PASETO v4.local (Arciszewski, 2021)	XChaCha20 + BLAKE2b-MAC (EtM)	None	Yes (by construction)	Inspired our deterministic-nonce + separate-MAC
HPKE-PQ draft 09 (Wood et al., 2026)	X-Wing (X25519 + ML-KEM-768)	Hybrid (KEM 0x647A)	No	The agility-camp answer to the same problem
MLS RFC 9420 (rfc, 2023)	X25519 MLS ciphersuite	Hybrid ML-KEM-768	No	Group messaging, not file
Signal PQXDH (Kret and Schmidt, 2023)	X25519 + Kyber-1024	Hybrid (legacy Kyber)	No	Messaging handshake
Apple iMessage PQ3 (Apple Security Engineering and Architecture, 2024)	ECDH P-256 + Kyber-1024 + rekey	Hybrid (legacy Kyber)	No	Production scale (∼1B devices)
AWS KMS PQ TLS (Amazon Web Services, 2024)	ML-KEM-	Transport hy-	N/A	Not an

12.2 KEM cost (Zen 4, cycles; sizes in bytes)

KEM	NIST	Keygen	Encaps	Decaps	pk	sk	ct	Source
ML-KEM-512	1	18 800	22 400	18 200	800	1 632	768	(PQ Code Packag
ML-KEM-768	3	30 100	35 900	29 400	1 184	2 400	1 088	(PQ Code Packag
ML-KEM-1024 (PQCrypt v1)	**5**	**46 200**	**54 800**	**45 300**	**1 568**	**3 168**	**1 568**	(PQ Code Packag
HQC-128	1	142 000	256 000	384 000	2 249	2 305	4 433	(HQC Submissior
Classic McEliece 348864	1	1.4×10^8	73 000	132 000	261 120	6 492	96	(Classic McEliece

12.3 AEAD throughput (1 MiB, single thread, GB/s)

AEAD	Sapphire R.	Zen 4	Apple M3	Notes
AES-256-GCM (PQCrypt v1)	**10.3**	**9.7**	**6.8**	AES-NI/VAES
AES-256-GCM-SIV	6.1	5.4	3.9	Extra GHASH pass
ChaCha20-Poly1305 (PQCrypt suite 0x02)	5.8	3.4	3.1	AVX-512 path on x86
AEGIS-128L	28.4	26.1	14.7	RFC 9678 Sep 2025
AEGIS-256	18.9	17.2	10.4	RFC 9678 Sep 2025
Ascon-128a (FIPS 232 final, Aug 2025)	0.55	0.52	0.61	Lightweight, constrained HW

Sources: (Bernstein and Lange, 2025; Intel Corporation, 2025; rfc, 2025). AEGIS is 2–3× faster than AES-256-GCM and is the principal candidate for a v1.x suite `0x03` once verified library implementations are available across all alpha languages.

12.4 Deployment scale (Nov 2026)

- Cloudflare Radar (Apr 2026): ∼38% of outbound TLS 1.3 to origins uses `X25519MLKEM768`; ∼32% of inbound from browsers (Cloudflare, 2026).
- Chrome: `X25519MLKEM768` default-on Chrome 131 (Nov 2024); `X25519Kyber768` fully removed in Chrome 141.
- OpenSSH 10.0 (Apr 2025): `mlkem768x25519-sha256` default key exchange.
- Apple iMessage PQ3: iOS 17.4 (Mar 2024), ∼1B devices.
- Signal PQXDH: mandatory client $\geq$ Sep 2023.
- AWS KMS PQ TLS to endpoints: GA Oct 2024.
- GCP Cloud KMS: PQ signing GA Feb 2026, PQ KEM preview.

The ecosystem reality: hybrid (X-Wing or TLS-style concat) has won the transport-protocol fight. PQCrypt is deliberately a counter-example for the envelope-format niche where unambiguous "PQ-only" is a feature, not a bug.

13 Appendices

A. Residual-risk acknowledgment for PQ-only

PQCrypt v1 ships one mandatory suite (ML-KEM-1024 + AES-256-GCM + UtC) and no hybrid. If ML-KEM-1024 is later structurally broken (a sub-exponential Module-LWE attack), every v1 envelope loses confidentiality with no classical fallback. This is a 30–50 year risk we accept in exchange for an unambiguous PQ-only promise. The forward-only mitigation path:

1. Future suite byte `0x06` introduces a hybrid suite (ML-KEM-1024 + X25519 + AES-256-GCM + UtC) once user demand signals.
2. Long-lived ciphertexts are re-encrypted under the v2 suite within a defined window.
3. v1 decoders remain available indefinitely (variant bytes are immutable post-allocation, matching the PQID and PQHash discipline).

We do *not* mitigate by hedging the v1 spec.

B. Variant byte allocation across pqtoolkit

Byte	Owner	Meaning	Status
`0x00`	—	Permanently reserved	MUST reject
`0x01`	PQID	Standard generator	assigned
`0x02`	PQID	HLC generator	assigned
`0x03`	PQHash	SHAKE-256 / 64 B tail	assigned
`0x04`	PQCrypt	Encryption envelope v1	assigned (this document)
`0x05`–`0xFE`	—	Reserved-future	MUST reject
`0xFF`	—	Permanently reserved	MUST reject

Cross-rejection is normative: PQID and PQHash decoders MUST reject `0x04`; PQCrypt decoders MUST reject `0x01`, `0x02`, `0x03`, and every byte in `0x05`–`0xFF` in v1. Each project's `invalid.json` conformance file pins the cross-rejection.

C. Suite byte allocation

Suite	Algorithms	Status
`0x01`	ML-KEM-1024 + AES-256-GCM + UtC commit + HKDF-SHA3-256	v1 mandatory
`0x02`	ML-KEM-1024 + ChaCha20-Poly1305 + UtC commit + HKDF-SHA3-256	v1 optional
`0x03`–`0xFF`	Reserved-future	MUST reject

Earmarked future suites (not normative in v1):

- `0x03`: ML-KEM-1024 + AEGIS-128L + UtC commit (once verified AEGIS implementations land).
- `0x06`: ML-KEM-1024 + X25519 + AES-256-GCM + UtC commit (hybrid).
- `0x07`: HQC + AES-256-GCM + UtC commit (lattice-fatigue backup).

D. Test-vector schema

Each KAT file is a JSON array of records. Common fields:

```
{
  "name":          "human-readable id, unique within the file",
  "op":            "encrypt | decrypt | peek_header | commit | ...",
  "input_hex":     "raw bytes for decode-from-binary ops",
  "input":         { ... op-specific structured input },
  "expected":      { ... op-specific structured output },
  "should_fail":   false,
  "error_kind":    "DECRYPT_FAILED | UNKNOWN_VARIANT | UNKNOWN_SUITE |
      INVALID_LEN"
}
```

Full schema in `pqcrypt/conformance/README.md`.

E. Acknowledgments

PQID and PQHash for the polyglot-spec-first discipline this whitepaper inherits. Bellare and Hoang for the UtC construction (Bellare and Hoang, 2022). The `mlkem-native` (Cryspen and PQ Code Package, 2025), RustCrypto `ml-kem`, and `filippo.io/mlkem` maintainers for the verified ML-KEM-1024 implementations PQCrypt v1 vendors. The KyberSlash (Bernstein et al., 2024), PQShield (PQShield, 2024), and GoFetch (Chen et al., 2024) disclosure teams, whose findings shape the side-channel disclaimer.

References

SHA-3 standard: Permutation-based hash and extendable-output functions. Technical Report FIPS PUB 202, National Institute of Standards and Technology, Aug. 2015. URL https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf.

Argon2 memory-hard function for password hashing and proof-of-work applications, 2021. URL https://datatracker.ietf.org/doc/html/rfc9106.

The messaging layer security (MLS) protocol, 2023. URL https://datatracker.ietf.org/doc/html/rfc9420.

Module-lattice-based key-encapsulation mechanism standard. Technical Report FIPS PUB 203, National Institute of Standards and Technology, Aug. 2024. URL https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf.

The AEGIS family of authenticated encryption algorithms, Sept. 2025. URL https://www.rfc-editor.org/rfc/rfc9678.

A. Albertini, T. Duong, S. Gueron, S. Kölbl, A. Luykx, and S. Schmieg. How to abuse and fix authenticated encryption without key commitment. IACR ePrint 2020/1456, 2020. URL https://eprint.iacr.org/2020/1456.

Amazon Web Services. Using post-quantum TLS with AWS KMS, 2024. URL https://docs.aws.amazon.com/kms/latest/developerguide/pqtls.html.

Apple Security Engineering and Architecture. imessage with PQ3: A new state of the art in quantum-secure messaging at scale, 2024. URL https://security.apple.com/blog/imessage-pq3/.

S. Arciszewski. PASETO v4.local specification, 2021. URL https://github.com/paseto-standard/paseto-spec/blob/master/docs/01-Protocol-Versions/Version4.md.

F. Bäumer, M. Dürmuth, and M. Brinkmann. Terrapin Attack: Breaking SSH channel integrity by sequence-number manipulation (CVE-2023-48795), 2023. URL https://terrapin-attack.com/.

M. Bellare and V. T. Hoang. Efficient schemes for committing authenticated encryption. In *Advances in Cryptology – EUROCRYPT 2022*, 2022. URL https://eprint.iacr.org/2022/1260.

D. J. Bernstein and T. Lange. eBACS: ECRYPT benchmarking of cryptographic systems (SUPERCOP), 2025. URL https://bench.cr.yp.to/.

D. J. Bernstein et al. KyberSlash 1 & 2: Timing-based key recovery on ML-KEM, 2024. URL https://kyberslash.cr.yp.to/.

P. Bose, V. T. Hoang, and S. Tessaro. Revisiting AES-GCM-SIV: Multi-user security, faster key derivation, and better bounds. IACR ePrint 2017/702, 2018. URL https://eprint.iacr.org/2017/702.

C2SP. The age plugin protocol, 2023. URL https://github.com/C2SP/C2SP/blob/main/age-plugin.md.

J. Chan and P. Rogaway. On committing authenticated encryption. IACR ePrint 2022/1260 (CTX construction), 2022. URL https://eprint.iacr.org/2022/1260.

B. Chen et al. GoFetch: Breaking constant-time cryptographic implementations using data memory-dependent prefetchers. USENIX Security 2024, 2024. URL https://gofetch.fail/.

Classic McEliece Submission Team. Classic McEliece: Implementation notes, 2022. URL https://classic.mceliece.org/mceliece-impl-20221023.pdf.

Cloudflare. Radar: Post-quantum TLS adoption, 2026. URL https://radar.cloudflare.com/adoption-and-usage.

Cryspen and PQ Code Package. mlkem-native: Formally verified ML-KEM for C/AArch64/AVX2, 2025. URL https://github.com/pq-code-package/mlkem-native.

J. Don, S. Fehr, C. Majenz, and C. Schaffner. Online-extractability in the quantum random-oracle model. In *EUROCRYPT 2022*, 2022. URL https://eprint.iacr.org/2021/280.

Google. Tink releases, 2026. URL https://github.com/tink-crypto/tink/releases.

Google Cloud. Post-quantum cryptography in Cloud KMS, 2026. URL https://cloud.google.com/kms/docs/post-quantum-cryptography.

P. Grubbs, J. Lu, and T. Ristenpart. Message franking via committing authenticated encryption. In *CRYPTO 2017*, 2017. URL https://eprint.iacr.org/2017/664.

Q. Guo, T. Johansson, and B.-Y. Yang. Single-trace SPA-CCA on unmasked ML-KEM on Cortex-M4. IACR ePrint 2024/2051, 2024. URL https://eprint.iacr.org/2024/2051.

V. T. Hoang and S. Tessaro. GCM-SST: AES-GCM with strong subkey-tagging. IACR ePrint 2024/875, 2024a. URL https://eprint.iacr.org/2024/875.

V. T. Hoang and S. Tessaro. Towards tight multi-user security of GCM. IACR ePrint 2024/875, 2024b. URL https://eprint.iacr.org/2024/875.

D. Hofheinz, K. Hövelmanns, and E. Kiltz. A modular analysis of the Fujisaki-Okamoto transformation. In *TCC 2017*, 2017. URL https://eprint.iacr.org/2017/604.

HQC Submission Team. HQC performance, 2025. URL https://pqc-hqc.org/doc/hqc-performance.pdf.

Intel Corporation. Data operand independent timing instructions (DOIT), 2022. URL https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/best-practices/data-operand-independent-timing-isa-guidance.html.

Intel Corporation. Intel ISA-L: Intelligent storage acceleration library, 2025. URL https://github.com/intel/isa-l_crypto.

E. Kret and R. Schmidt. The PQXDH key agreement protocol, 2023. URL https://signal.org/docs/specifications/pqxdh/.

J. Len, P. Grubbs, and T. Ristenpart. Partitioning oracle attacks. USENIX Security 2021, 2021. URL https://eprint.iacr.org/2020/1153.

PQ Code Package. mlkem-native benchmarks, 2026. URL https://github.com/pq-code-package/mlkem-native#benchmarks.

PQShield. Plugging timing leaks in Kyber/ML-KEM via masking, 2024. URL https://pqshield.com/blog/power-side-channel-ml-kem.

F. Valsorda and contributors. age releases (filippo.io/age), 2025. URL https://github.com/FiloSottile/age/releases.

Y. Wang, R. Paccagnella, E. T. He, H. Shacham, C. W. Fletcher, and D. Kohlbrenner. Hertzbleed: Turning power side-channel attacks into remote timing attacks on x86. USENIX Security 2022, 2022. URL https://www.hertzbleed.com/.

C. A. Wood et al. Hybrid HPKE for post-quantum KEMs (draft-ietf-hpke-pq-09), 2026. URL https://datatracker.ietf.org/doc/draft-ietf-hpke-pq/.